

FACULDADE DE ENSINO SUPERIOR DO PARANÁ
BACHARELADO EM SISTEMAS DE INFORMAÇÃO
PROGRAMAÇÃO ORIENTADA A OBJETOS

BEM WMS
WAREHOUSE MANAGEMENT SYSTEM DE CONTROLE DE ENTRADA E SAÍDA
DE ESTOQUE

Emily Kristin Garcia
Bruno Batista Xavier
Maria Eduarda Martins de Souza

Curitiba
2026

1. TÍTULO DO PROJETO

BEM WMS - Warehouse Management System de Controle de Entrada e Saída de Estoque.

2. NOME DOS INTEGRANTES

Integrantes do grupo:

- Emily Kristin Garcia
- Bruno Batista Xavier
- Maria Eduarda Martins de Souza

3. IDENTIFICAÇÃO ACADÊMICA

Campo	Informação
Instituição de ensino	Faculdade de Ensino Superior do Paraná
Cidade	Curitiba
Curso	Bacharelado em Sistemas de Informação
Matéria	Programação Orientada a Objetos

4. OBJETIVO DO PROJETO

O objetivo do projeto é desenvolver um sistema de controle de estoque para materiais escolares, permitindo o gerenciamento de entradas, saídas, lotes, status dos materiais e indicadores financeiros.

O BEM WMS foi criado para simular um sistema de armazém/estoque, no qual é possível cadastrar materiais escolares, consultar itens cadastrados, listar materiais ativos e inativos, editar registros, remover materiais e registrar saídas de estoque por lote.

Além das funcionalidades básicas de uma API, o sistema também possui uma interface web em modo dark, desenvolvida com HTML, CSS e JavaScript, permitindo que o usuário interaja com os dados de forma visual e intuitiva.

O sistema também apresenta um dashboard financeiro, exibindo informações como valor investido, venda estimada, lucro estimado e ROI, possibilitando uma visão mais completa sobre o estoque.

A proposta da equipe foi criar uma aplicação simples, funcional e organizada, utilizando ASP.NET Core Minimal API no back-end e um front-end consumindo os dados da API por meio de requisições HTTP.

5. ESTRUTURA DA SOLUÇÃO

A entidade principal do sistema é o MaterialEscolar.

Atributo	Tipo	Descrição
Id	int	Identificador único do material
Nome	string	Nome do material escolar
Categoria	string	Categoria do material
Ativo	bool	Indica se o material está ativo ou inativo
Lotes	List<Lote>	Lista de lotes vinculados ao material

Além disso, existe a entidade Lote, que representa os lotes associados aos materiais escolares.

Atributo	Tipo	Descrição
Id	int	Identificador único do lote
Codigo	string	Código do lote
Quantidade	int	Quantidade de itens disponíveis no lote
DataEntrada	DateTime	Data de entrada do lote no estoque
DataVencimento	DateTime?	Data de vencimento do lote, quando existir
ValorCusto	decimal	Valor de custo unitário do item
ValorVenda	decimal	Valor de venda unitário do item

Também existe a entidade SaidaEstoque, utilizada para registrar a saída de materiais.

Atributo	Tipo	Descrição
MaterialId	int	Identificador do material que terá saída
Quantidade	int	Quantidade que será retirada do estoque

A API se relaciona com os dados por meio de uma lista simulada em memória, localizada no arquivo BancoSimulado.cs. Essa abordagem foi utilizada para fins acadêmicos e demonstração do funcionamento da API, sem conexão direta com um banco de dados.

6. FUNCIONALIDADES DO SISTEMA

- Cadastro de materiais escolares;

- Cadastro de dados de lote;
- Consulta de todos os materiais;
- Consulta de materiais ativos;
- Consulta de materiais inativos;
- Consulta de material por ID;
- Consulta de lotes por material;
- Edição de materiais cadastrados;
- Remoção de materiais;
- Registro de saída de estoque;
- Controle de saída por lote;
- Dashboard financeiro;
- Cálculo de valor investido;
- Cálculo de venda estimada;
- Cálculo de lucro estimado;
- Cálculo de ROI;
- Tabela de itens, códigos e lotes;
- Interface web em modo dark;
- Consumo da API pelo front-end.

7. REGRA DE SAÍDA POR LOTE

O sistema realiza a saída de estoque considerando os lotes disponíveis para cada material. Quando o material possui data de vencimento, o sistema prioriza os lotes com vencimento mais próximo. Essa regra é conhecida como FEFO - First Expire, First Out, ou seja, primeiro que vence, primeiro que sai.

Para materiais sem data de vencimento, o sistema utiliza a data de entrada como critério de ordenação, priorizando os lotes mais antigos.

8. CÁLCULOS FINANCEIROS

O sistema possui um dashboard que apresenta informações financeiras do estoque.

Valor Investido = Quantidade em Estoque x Valor de Custo Unitário

Venda Estimada = Quantidade em Estoque x Valor de Venda Unitário

Lucro Estimado = Venda Estimada - Valor Investido

$$\text{ROI (\%)} = (\text{Lucro Estimado} / \text{Valor Investido}) \times 100$$

Exemplo: considerando 50 unidades de um item com custo unitário de R\$ 8,50 e venda unitária de R\$ 14,90, o valor investido é R\$ 425,00, a venda estimada é R\$ 745,00, o lucro estimado é R\$ 320,00 e o ROI é 75,29%.

9. ENDPOINTS DA API

Método	Rota	Descrição
GET	/	Retorna uma mensagem informando que a API está funcionando
GET	/api/materiais	Lista todos os materiais cadastrados
GET	/api/materiais/{id}	Busca um material específico pelo identificador
GET	/api/materiais/{id}/lotes	Lista os lotes vinculados a um material
GET	/api/materiais/ativos	Lista apenas os materiais ativos
GET	/api/materiais/inativos	Lista apenas os materiais inativos
POST	/api/materiais	Cadastra um novo material
PUT	/api/materiais/{id}	Edita um material cadastrado
DELETE	/api/materiais/{id}	Remove um material pelo identificador
POST	/api/materiais/saida	Registra a saída de estoque de um material
GET	/api/dashboard	Retorna os indicadores financeiros e operacionais do sistema

10. DETALHAMENTO DOS ENDPOINTS

GET /

Endpoint utilizado para verificar se a API está funcionando corretamente. O retorno esperado é: API de Materiais Escolares funcionando!

GET /api/materiais

Retorna todos os materiais cadastrados na lista simulada, incluindo seus respectivos lotes, quantidades, códigos, datas e valores.

GET /api/materiais/{id}

Busca um material pelo seu Id. Caso o material não seja encontrado, retorna a mensagem: Material não encontrado.

GET /api/materiais/{id}/lotes

Retorna todos os lotes vinculados a um material específico.

GET /api/materiais/ativos

Retorna somente os materiais em que o campo Ativo está definido como true.

GET /api/materiais/inativos

Retorna somente os materiais em que o campo Ativo está definido como false.

POST /api/materiais

Permite cadastrar um novo material na lista simulada. O corpo da requisição deve conter os dados do material e seu respectivo lote.

PUT /api/materiais/{id}

Permite editar um material já cadastrado. A rota atualiza os dados do material, como nome, categoria, status e informações do lote.

DELETE /api/materiais/{id}

Remove um material da lista com base no seu Id. Caso o material exista, ele é removido com sucesso.

POST /api/materiais/saida

Registra a saída de estoque de um material. A rota valida a quantidade solicitada, verifica se há estoque suficiente e reduz a quantidade dos lotes disponíveis.

GET /api/dashboard

Retorna os indicadores operacionais e financeiros do sistema, como total de materiais, ativos, inativos, unidades em estoque, valor investido, venda estimada, lucro estimado e ROI percentual.

11. FRONT-END

Além da API, o projeto possui uma interface web desenvolvida com HTML, CSS e JavaScript. A interface possui a nomenclatura BEM WMS - Warehouse Management System de Controle de Entrada e Saída de Estoque.

O front-end possui as seguintes seções: dashboard, cadastro de material, registro de saída de estoque, tabela de itens, códigos e lotes, e lista de materiais cadastrados.

12. FUNCIONALIDADES DO FRONT-END

A seção de dashboard apresenta os principais indicadores do sistema: total de materiais, itens ativos, itens inativos, unidades em estoque, valor investido, venda estimada, lucro estimado e ROI.

O dashboard financeiro por material apresenta uma tabela com material, categoria, estoque, valor investido, venda estimada, lucro e ROI.

O formulário de cadastro permite inserir o ID do material, nome, categoria, status, dados do lote, quantidade em estoque, valor de custo, valor de venda, data de entrada e data de vencimento.

Na listagem de materiais, o botão Editar material carrega os dados do item no formulário. Após a edição, o sistema salva as alterações utilizando a rota PUT /api/materiais/{id}.

A seção de saída permite selecionar um material e informar a quantidade que será retirada do estoque. Após registrar a saída, o sistema atualiza automaticamente a quantidade em estoque, o dashboard, a tabela de itens e a lista de materiais.

A tabela de itens, códigos e lotes exibe ID do material, material, categoria, status, ID do lote, código do lote, quantidade, data de entrada, data de vencimento, custo unitário e venda unitária.

13. ORGANIZAÇÃO DO CÓDIGO

O projeto foi organizado em pastas e arquivos separados para facilitar a manutenção e a leitura do código.

Pasta Models

A pasta Models contém as classes MaterialEscolar.cs, Lote.cs e SaidaEstoque.cs, responsáveis por representar as entidades principais do sistema.

Pasta Data

A pasta Data contém o arquivo BancoSimulado.cs, responsável por armazenar os materiais simulados em memória. Na versão atual, a base simulada pode gerar uma quantidade maior de materiais para demonstração.

Pasta Routes

A pasta Routes contém os arquivos responsáveis pelas rotas da API: ROTA_GET.cs, ROTA_GET_ATIVOS.cs, ROTA_GET_INATIVOS.cs, ROTA_POST.cs, ROTA_PUT.cs, ROTA_DELETE.cs, ROTA_SAIDA.cs e ROTA_DASHBOARD.cs.

Program.cs

O arquivo Program.cs é o ponto de entrada da aplicação. Ele configura o Swagger, o CORS, registra as rotas da API e inicializa a execução do sistema.

As rotas são registradas por meio dos métodos: app.MapGetRoutes();
app.MapGetAtivosRoutes(); app.MapGetInativosRoutes(); app.MapPostRoutes();
app.MapPutRoutes(); app.MapDeleteRoutes(); app.MapSaidaRoutes();
app.MapDashboardRoutes();

Pasta frontend

A pasta frontend contém os arquivos index.html, style.css e script.js. O arquivo index.html estrutura a tela, o style.css define o layout em modo dark e o script.js realiza a comunicação com a API por meio de fetch.

14. ARQUIVOS DE CONFIGURAÇÃO

O arquivo appsettings.json armazena configurações gerais da aplicação. O appsettings.Development.json armazena configurações específicas do ambiente de desenvolvimento. O launchSettings.json, localizado na pasta Properties, armazena configurações de inicialização do projeto durante o desenvolvimento.

15. COMO EXECUTAR O PROJETO

Back-end

No terminal, acesse a pasta do projeto onde está o arquivo .csproj:

```
cd C:\Users\emily.garcia\WMS_API\WAREHOUSESYSTEM_BACKEND\ApiProdutos
```

Restaure as dependências com: dotnet restore

Execute a API com: `dotnet run`

A API será iniciada em uma URL semelhante a `http://localhost:5181`. O Swagger pode ser acessado em `http://localhost:5181/swagger`.

Front-end

Acesse a pasta frontend no VS Code e abra o arquivo `index.html` com a extensão Live Server. A URL será semelhante a `http://127.0.0.1:5500/frontend/index.html`.

No arquivo `script.js`, a URL da API deve estar configurada corretamente por meio da constante `API_URL`. Caso a API execute em outra porta, é necessário atualizar essa constante.

16. JUSTIFICATIVA TÉCNICA

A equipe optou por desenvolver a API utilizando ASP.NET Core Minimal API, pois essa abordagem permite criar endpoints de forma simples, rápida e objetiva, sem a necessidade de estruturas mais complexas como controllers tradicionais.

A separação das rotas em arquivos diferentes também foi uma decisão técnica importante, pois melhora a organização do projeto e facilita a manutenção do código. Dessa forma, cada arquivo possui uma responsabilidade específica, como consulta, cadastro, edição, exclusão, saída de estoque e dashboard.

Outra decisão adotada foi a utilização de uma lista simulada em memória para armazenar os dados. Essa escolha foi feita por se tratar de um projeto acadêmico, com foco principal na criação e funcionamento dos endpoints da API. Em um ambiente real, essa estrutura poderia ser evoluída para utilizar um banco de dados, como SQL Server, PostgreSQL, MySQL ou SQLite.

Também foi implementado o Swagger, que facilita a visualização, teste e documentação dos endpoints da API. Com ele, é possível testar as rotas diretamente pelo navegador, tornando o desenvolvimento e a apresentação do projeto mais simples.

O front-end foi desenvolvido com HTML, CSS e JavaScript puro, permitindo consumir os endpoints da API de forma direta com `fetch`. A interface em modo dark foi escolhida para melhorar a visualização e trazer uma aparência mais moderna ao sistema.

A criação das entidades MaterialEscolar, Lote e SaidaEstoque permite representar uma estrutura próxima de um sistema real de estoque, onde cada material pode possuir diferentes lotes, quantidades, datas de entrada, datas de vencimento e valores financeiros.

A implementação da saída por lote torna a solução mais próxima de um WMS real, pois considera regras de controle de estoque como a priorização de lotes com vencimento mais próximo.

A inclusão do dashboard financeiro amplia a proposta do sistema, permitindo que o usuário visualize não apenas a quantidade de itens em estoque, mas também indicadores como valor investido, venda estimada, lucro estimado e ROI.

17. POSSÍVEIS MELHORIAS FUTURAS

- Integração com banco de dados;
- Histórico de movimentações de entrada e saída;
- Cadastro de múltiplos lotes por material;
- Edição individual de lotes;
- Controle de usuários;
- Login e autenticação;
- Níveis de permissão;
- Alertas de estoque baixo;
- Alertas de vencimento próximo;
- Relatórios exportáveis;
- Dashboard com gráficos;
- Integração com Power BI;
- Melhorias de acessibilidade;
- Responsividade avançada;
- Deploy da API e do front-end.